

MANUAL DE REFERENCIA

Python Core

Manual de consulta y repaso

Repaso estructurado de fundamentos, colecciones, funciones, POO, errores, archivos y módulos clave.

Propósito: Este manual está pensado como documento de repaso y consulta. No pretende reemplazar una ruta de aprendizaje desde cero: organiza, amplía y precisa los conceptos centrales de Python para volver a ellos con rapidez y rigor.

Criterio editorial: El manual prioriza precisión conceptual, consulta rápida y claridad terminológica. Incluye notas de precisión, tablas de consulta, módulos y conceptos clave para repasar con rigor.

Referencia estructurada para Python 3.x

Cómo usar este manual

Este documento funciona mejor como mapa de consulta: puedes leerlo completo una vez y luego regresar a secciones concretas cuando necesites refrescar sintaxis, diferencias conceptuales o detalles técnicos.

- Para repasar fundamentos: secciones 1 a 9.
- Para estructuras de datos: secciones 10 a 13.
- Para diseño de código: secciones 14 a 17.
- Para robustez y trabajo con recursos: secciones 18 a 21.
- Para aclarar vocabulario técnico: sección 22, Glosario.

Lectura recomendada: Si ya programas con comodidad, una ruta eficiente es leer primero las notas de precisión, revisar tablas de métodos y dejar el glosario para consolidar términos.

Contenido

1. Objetos, nombres, variables y tipos	3
2. Tipos de datos básicos y conversiones	4
3. Entrada, salida y representación textual	5
4. Operadores, matemáticas y precisión numérica	6
5. Condicionales, verdad lógica y comparaciones	7
6. Cadenas, Unicode y métodos de texto	8
7. Indexación, slicing y desempaqueado	9
8. F-strings y especificadores de formato	10
9. Bucles, iteración y control de flujo	11
10. Colecciones: listas, tuplas, conjuntos y range	12
11. Diccionarios y mapeos	14
12. Mutabilidad, aliasing y copias	15
13. Comprensiones, iterables e iteradores	16
14. Funciones, parámetros y modularidad	17
15. Scope, namespaces y regla LEGB	18
16. Módulos, imports, paquetes y __main__	19
17. Programación orientada a objetos	19
18. Excepciones y gestión de errores	21
19. Archivos, rutas y sistema operativo	22
20. Módulos estándar clave	23
21. Buenas prácticas y trampas frecuentes	25
22. Glosario técnico	26
23. Fuentes oficiales consultadas	28

1. Objetos, nombres, variables y tipos

Python se entiende mejor si se piensa en objetos y referencias. Una variable no es una caja fija con un tipo inmutable: es un nombre que queda asociado a un objeto.

Modelo mental mínimo

Concepto	Descripción precisa
Objeto	Entidad existente en memoria con identidad, tipo y valor. En Python, números, cadenas, listas, funciones y clases son objetos.
Nombre o identificador	Etiqueta que el programa usa para referirse a un objeto. Una variable es un nombre enlazado a un objeto.
Asignación	Operación que vincula un nombre con un objeto. No implica necesariamente copiar datos.
Tipo	Categoría que determina operaciones, comportamiento y representación de un objeto.
Identidad	Rasgo que distingue un objeto concreto de otro. Puede consultarse con <code>id()</code> en CPython y otras implementaciones.

Término técnico - referencia: Una referencia es el vínculo que permite que un nombre apunte a un objeto. Dos nombres pueden referirse al mismo objeto mutable.

Sintaxis conceptual

```
a = [1, 2, 3]
b = a
# a y b se refieren a la misma lista
```

Tipado dinámico

Python es dinámicamente tipado: el nombre no queda bloqueado a un solo tipo de dato. Lo que tiene tipo es el objeto al que apunta en cada momento.

Expresión	Idea
<code>x = 10</code>	x referencia un entero.
<code>x = 'diez'</code>	El mismo nombre x puede luego referenciar una cadena.
<code>type(x)</code>	Permite inspeccionar el tipo del objeto referenciado.

Precisión: Que Python sea dinámico no significa que sea débilmente tipado. Operaciones incompatibles, como sumar directamente una cadena y un entero, producen `TypeError`.

2. Tipos de datos básicos y conversiones

Los tipos integrados constituyen el vocabulario elemental de Python. Conviene distinguir tipos numéricos, texto, valores lógicos, ausencia de valor y secuencias binarias.

Tipos principales

Tipo	Sintaxis habitual	Idea central
str	'texto' o "texto"	Cadena inmutable de texto Unicode.
int	42	Entero de precisión arbitraria, limitado en la práctica por memoria y restricciones de implementación.
float	3.14	Número de punto flotante de doble precisión; representa aproximaciones binarias de muchos decimales.
complex	2 + 3j	Número complejo con parte real e imaginaria.
bool	True / False	Valores lógicos; bool es un subtipo de int, aunque semánticamente se usa para verdad o falsedad.
NoneType	None	Objeto único que expresa ausencia intencional de valor.
bytes	b'abc'	Secuencia inmutable de bytes, no de caracteres textuales.
bytearray	bytearray(b'abc')	Secuencia mutable de bytes.

Término técnico - Unicode: Unicode es un estándar que asigna números llamados code points a caracteres de muchos sistemas de escritura. En Python, str representa texto Unicode, no una secuencia cruda de bytes.

Conversión e inspección

Función	Qué hace	Notas útiles
type(obj)	Retorna el tipo del objeto.	Útil para inspección, aunque para verificar jerarquías suele preferirse isinstance().
str(obj)	Obtiene representación textual orientada a lectura.	No siempre coincide con repr(obj), que suele ser más explícita.
int(obj)	Construye un entero desde número o texto válido.	Con float trunca hacia cero: descarta la parte fraccionaria sin redondear.
float(obj)	Construye un float desde entero o texto numérico válido.	La representación puede no ser exacta para decimales como 0.1.

Función	Qué hace	Notas útiles
<code>bool(obj)</code>	Evalúa la verdad lógica del objeto.	False para None, False, ceros y colecciones vacías; True para la mayoría de los demás objetos.
<code>isinstance(obj, tipo)</code>	Comprueba si el objeto pertenece a un tipo o a su jerarquía.	Suele ser más flexible que comparar <code>type(obj) == tipo</code> .

Término técnico - truncar: Truncar es cortar la parte fraccionaria sin redondear. `int(3.9)` produce 3 e `int(-3.9)` produce -3: el acercamiento es hacia cero.

Verdad lógica de objetos

Se considera False	Ejemplos
Ausencia y falso lógico	None, False
Ceros numéricos	0, 0.0, 0j
Colecciones vacías	", 0, [], {}, set(), range(0)

Fuera de esos casos típicos, la mayoría de los objetos se evalúan como verdaderos, salvo que su clase defina reglas especiales de verdad mediante `__bool__()` o `__len__()`.

3. Entrada, salida y representación textual

La interacción con consola descansa sobre `input()` y `print()`, pero su uso correcto implica conocer retornos, separadores, finales de línea y representación de valores.

input y print

Elemento	Sintaxis	Comportamiento
Entrada	<code>input(prompt)</code>	Muestra un mensaje opcional, lee una línea y retorna siempre str.
Salida	<code>print(*objects, sep=' ', end='\n')</code>	Imprime uno o varios objetos, separados por sep y finalizados con end.

Recordatorio sintáctico

```
nombre = input('Nombre: ')
print('Hola', nombre, sep=' -> ', end='!\n')
```

str frente a repr

Función	Intención	Uso típico
str(obj)	Representación legible para usuario.	Mensajes, reportes y salida amigable.
repr(obj)	Representación más precisa o inequívoca para depuración.	Inspección de strings con comillas, objetos y debugging.

Precisión: input() no convierte automáticamente a int o float. Si necesitas un número, debes convertir explícitamente y manejar ValueError cuando la entrada pueda ser inválida.

4. Operadores, matemáticas y precisión numérica

Los operadores aritméticos parecen directos, pero hay diferencias importantes entre división real, división piso, truncamiento y redondeo.

Operadores aritméticos

Operación	Sintaxis	Descripción
Suma / resta	$a + b$ / $a - b$	Aritmética elemental. + también concatena secuencias compatibles.
Multiplicación	$a * b$	Producto numérico o repetición de secuencias.
División real	a / b	Retorna cociente de tipo float en operaciones entre enteros.
División piso	$a // b$	Retorna el cociente redondeado hacia abajo; con negativos no equivale a truncar hacia cero.
Módulo	$a \% b$	Resto coherente con la división piso: $a == (a // b) * b + (a \% b)$.
Potencia	$a ** b$	Eleva a a la potencia b.
Asignación compuesta	$a += b$, $a -= b$	Opera y reasigna; en mutables el efecto puede depender del tipo.

Precisión: $-3 // 2$ da -2, no -1. La división // aplica piso matemático, mientras que $\text{int}(-1.5)$ trunca hacia cero y da -1.

Funciones matemáticas frecuentes

Función	Idea
round(x, n)	Redondea a n decimales. En empates exactos usa redondeo al par. Los floats pueden dar resultados sorprendentes por representación binaria.

Función	Idea
<code>abs(x)</code>	Valor absoluto o magnitud.
<code>pow(base, exp)</code>	Potencia integrada; equivalente básico a <code>**</code> .
<code>max(...)</code> / <code>min(...)</code>	Máximo o mínimo de argumentos o de un iterable.
<code>sum(iterable)</code>	Suma elementos numéricos de un iterable.
<code>divmod(a, b)</code>	Retorna cociente y resto como par.

Módulo math

Elemento	Uso
<code>math.pi</code>	Constante pi.
<code>math.e</code>	Constante de Euler.
<code>math.sqrt(x)</code>	Raíz cuadrada.
<code>math.ceil(x)</code>	Techo: menor entero $\geq x$.
<code>math.floor(x)</code>	Piso: mayor entero $\leq x$.
<code>math.trunc(x)</code>	Truncamiento de la parte fraccionaria hacia cero.

Punto flotante: idea esencial

Un float no almacena todos los decimales de forma exacta. Muchos valores decimales se aproximan internamente en base 2. Por eso, ciertos cálculos presentan pequeñas diferencias y conviene evitar comparar floats por igualdad exacta cuando provienen de operaciones acumuladas.

Término técnico - precisión: Precisión es la capacidad de representar o distinguir valores con cierto detalle. En floats, precisión no significa exactitud decimal ilimitada.

5. Condicionales, verdad lógica y comparaciones

La ejecución condicional en Python depende de expresiones que se evalúan como verdaderas o falsas. Esto abarca comparaciones, operadores lógicos, pertenencia e identidad.

Estructura condicional

Forma general

```
if condicion_1:
    # bloque principal
elif condicion_2:
    # alternativa
else:
    # caso por defecto
```

Comparación, lógica, pertenencia e identidad

Familia	Operadores	Qué expresan
Comparación	<code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	Igualdad, desigualdad y orden cuando aplica.
Lógica	<code>and</code> , <code>or</code> , <code>not</code>	Composición de condiciones con evaluación de cortocircuito.
Pertenencia	<code>in</code> , <code>not in</code>	Indica si un elemento pertenece a una colección; en dict prueba claves.
Identidad	<code>is</code> , <code>is not</code>	Comprueba si dos nombres se refieren al mismo objeto, no si sus valores son equivalentes.

Precisión: Usa `==` para comparar valores y `is` para identidad. La comparación con `None` se escribe normalmente como `x is None`.

Expresión condicional

Sintaxis ternaria

```
valor_si_verdadero if condicion else valor_si_falso
```

Úsala cuando la decisión cabe en una expresión breve. Si la lógica se vuelve densa, un `if` convencional suele ser más legible.

6. Cadenas, Unicode y métodos de texto

Las cadenas `str` son secuencias inmutables de texto Unicode. Entender esta frase evita muchos errores con archivos, acentos, emojis, bytes y codificaciones.

Texto, caracteres y codificación

Término	Explicación
Unicode	Estándar de representación abstracta de caracteres mediante code points.
Code point	Número asignado por Unicode a una unidad abstracta, por ejemplo <code>U+0041</code> para la A latina mayúscula.
Glifo	Forma visual concreta que una fuente dibuja para un carácter o combinación de caracteres.
Codificación	Reglas para convertir texto Unicode a bytes, como UTF-8.
UTF-8	Codificación muy extendida que representa code points usando uno o más bytes.

Precisión: str y bytes no son intercambiables. str representa texto; bytes representa datos binarios. encode() transforma texto en bytes y decode() realiza el camino inverso.

Métodos de cadenas

Método / función	Efecto principal	Detalle útil
len(s)	Longitud en elementos de la cadena.	Cuenta unidades de la secuencia de str, no ancho visual en pantalla.
s.find(sub)	Índice de la primera coincidencia o -1.	No lanza error si no encuentra.
s.rfind(sub)	Índice de la última coincidencia o -1.	Busca desde la derecha.
s.capitalize()	Primera letra en mayúscula y resto en minúscula.	Retorna cadena nueva.
s.upper() / s.lower()	Conversión de mayúsculas/minúsculas.	No modifica la cadena original.
s.casefold()	Normalización más agresiva para comparación insensible a mayúsculas.	Útil en texto Unicode.
s.isdigit()	True si todos los caracteres son dígitos según Unicode.	No equivale estrictamente a 'solo 0-9 ASCII'.
s.isalpha()	True si todos los caracteres son alfabéticos y la cadena no está vacía.	Puede aceptar letras acentuadas y de otros alfabetos.
s.count(sub)	Cuenta ocurrencias no superpuestas.	Retorna entero.
s.replace(old, new)	Sustituye segmentos.	Retorna cadena nueva.
sep.join(iterable)	Une strings usando sep.	Los elementos deben ser strings.
s.split(sep)	Divide una cadena en partes.	Si sep se omite, separa por espacios en blanco.
s.strip()	Elimina bordes de espacios o caracteres indicados.	No toca el centro del texto.

Término técnico - inmutable: Un objeto inmutable no puede modificarse en el mismo lugar una vez creado. Los métodos de str retornan nuevas cadenas en lugar de alterar la original.

7. Indexación, slicing y desempaquetado

Las secuencias permiten acceder a elementos individuales y a fragmentos. El slicing es una notación compacta y muy poderosa.

Indexación

Notación	Significado
secuencia[0]	Primer elemento.

Notación	Significado
secuencia[-1]	Último elemento.
secuencia[-2]	Penúltimo elemento.

Slicing

Forma general

```
secuencia[inicio:fin:paso]
```

Parte	Regla
inicio	Índice inclusivo. Si se omite, se toma el extremo inicial adecuado al paso.
fin	Índice exclusivo. El elemento en esa posición no se incluye.
paso	Salto entre posiciones. Un paso negativo recorre en sentido inverso.

Ejemplo	Lectura
s[1:4]	Elementos desde índice 1 hasta antes del 4.
s[:3]	Primeros tres elementos.
s[::2]	Elementos alternados.
s[::-1]	Secuencia invertida.

Desempaquetado

Asignación por desempaquetado

```
a, b = (10, 20)
primero, *medio, ultimo = [1, 2, 3, 4]
```

El operador `*` en asignaciones permite capturar una cantidad variable de elementos intermedios. Es frecuente al trabajar con tuplas, listas o resultados de funciones.

8. F-strings y especificadores de formato

Las f-strings permiten interpolar expresiones dentro de texto y controlar de forma precisa cómo se muestra un valor.

Forma general

F-string

```
f"texto {expresion:especificador}"
```

Especificadores frecuentes

Especificador	Efecto
::2f	Muestra dos decimales y redondea para representación.
:8	Reserva ancho mínimo de ocho caracteres.
:08	Rellena con ceros a la izquierda hasta ancho ocho.
:<10	Alinea a la izquierda.
:>10	Alinea a la derecha.
:^10	Centra el contenido.
:+	Fuerza signo positivo o negativo en números.
:,	Inserta separador de miles con coma.
:_	Inserta guion bajo como separador de miles, útil en algunos contextos técnicos.

Precisión: Formatear un float no cambia su valor interno; solo cambia su representación textual al mostrarse.

9. Bucles, iteración y control de flujo

Los bucles recorren datos o repiten procesos bajo una condición. Python separa con claridad el while condicional del for sobre iterables.

while y for

Estructura	Cuándo usarla
while condicion	Cuando la repetición depende de una condición cuya duración no se conoce de antemano.
for variable in iterable	Cuando se recorren elementos de un objeto iterable.

range y utilidades de iteración

Elemento	Uso
range(start, stop, step)	Genera una secuencia inmutable de enteros para iterar.
reversed(seq)	Retorna un iterador que recorre la secuencia en sentido inverso.
enumerate(iterable, start=0)	Produce pares índice-elemento.
zip(a, b, ...)	Agrupar elementos de varios iterables por posición.

Control de ciclo

Sentencia	Efecto
break	Sale inmediatamente del bucle actual.
continue	Salta al inicio de la siguiente iteración.
else en bucles	Se ejecuta si el bucle termina sin pasar por break.

Precisión: El else de un for o while no significa 'si la condición fue falsa' en sentido cotidiano; significa 'si el bucle terminó de forma normal, sin break'.

10. Colecciones: listas, tuplas, conjuntos y range

Las colecciones agrupan múltiples objetos. La diferencia esencial entre ellas se entiende con cuatro preguntas: ¿mantienen orden?, ¿permiten duplicados?, ¿son mutables?, ¿se accede por índice?

Comparación general

Tipo	Orden	Duplicados	Mutabilidad	Indexación
list	Sí	Sí	Mutable	Sí
tuple	Sí	Sí	Inmutable	Sí
set	Sin orden de indexación contractual	No	Mutable	No
frozenset	Sin orden de indexación contractual	No	Inmutable	No
range	Secuencia ordenada de enteros	Según la secuencia	Inmutable	Sí

Precisión: Que una colección muestre cierta disposición al imprimirla no equivale a que garantice un orden indexable. Los sets no deben usarse cuando el orden sea parte de la lógica.

Métodos de listas

Método	Efecto
append(x)	Añade x al final.
extend(iterable)	Añade todos los elementos del iterable.
insert(i, x)	Inserta x en la posición i.
remove(x)	Elimina la primera ocurrencia de x; error si no existe.
pop(i=-1)	Elimina y retorna el elemento de la posición indicada; por defecto, el último.

Método	Efecto
<code>clear()</code>	Vacía la lista.
<code>index(x)</code>	Retorna la posición de la primera ocurrencia de x.
<code>count(x)</code>	Cuenta ocurrencias de x.
<code>sort(...)</code>	Ordena in-place.
<code>reverse()</code>	Invierte in-place.
<code>copy()</code>	Retorna copia superficial de la lista.

Precisión: Los métodos de listas que modifican in-place y no deben devolver un valor útil retornan None. Por eso `lista = lista.sort()` rompe el contenido esperado de lista.

Tuplas

Las tuplas son colecciones ordenadas e inmutables. Son útiles para representar agrupaciones fijas, claves de diccionarios cuando sus elementos son hashables y retornos múltiples de funciones.

Método	Efecto
<code>tuple.index(x)</code>	Posición de la primera coincidencia.
<code>tuple.count(x)</code>	Número de ocurrencias.

Precisión: La inmutabilidad de la tupla se refiere a su estructura. Una tupla puede contener un objeto mutable que sí cambie internamente.

Conjuntos

Método / operación	Efecto
<code>add(x)</code>	Añade x si no estaba.
<code>remove(x)</code>	Elimina x; error si no existe.
<code>discard(x)</code>	Elimina x si existe; no falla si falta.
<code>pop()</code>	Elimina y retorna un elemento arbitrario.
<code>clear()</code>	Vacía el conjunto.
<code>a b / a.union(b)</code>	Unión.
<code>a & b / a.intersection(b)</code>	Intersección.
<code>a - b / a.difference(b)</code>	Diferencia.

Colecciones bidimensionales

Una colección bidimensional suele representarse como una lista de listas o tupla de tuplas. El acceso encadenado `coleccion[fila][columna]` primero selecciona una subcolección y luego el elemento interno.

11. Diccionarios y mapeos

Un diccionario asocia claves con valores. Es la estructura central de mapeo en Python y se usa para modelar registros, configuraciones, conteos y datos estructurados.

Estructura

Sintaxis general

```
diccionario = {clave_1: valor_1, clave_2: valor_2}
```

Propiedad	Descripción
Claves	Deben ser hashables, normalmente inmutables.
Valores	Pueden ser de cualquier tipo.
Orden	Los diccionarios preservan el orden de inserción en Python moderno.
Acceso	<code>dict[clave]</code> busca de forma directa; lanza <code>KeyError</code> si la clave no existe.

Término técnico - hashable: Un objeto hashable tiene un valor hash estable durante su vida útil y puede utilizarse como clave en `dict` o elemento de set. Las listas no son hashables; las tuplas pueden serlo si sus elementos también lo son.

Métodos principales

Método	Efecto
<code>get(key, default=None)</code>	Acceso seguro; retorna <code>default</code> si la clave falta.
<code>update(...)</code>	Inserta o sobrescribe pares clave-valor.
<code>pop(key[, default])</code>	Elimina clave y retorna su valor.
<code>popitem()</code>	Elimina y retorna el último par insertado.
<code>clear()</code>	Vacía el diccionario.
<code>keys()</code>	Vista dinámica de claves.
<code>values()</code>	Vista dinámica de valores.
<code>items()</code>	Vista dinámica de pares (clave, valor).
<code>setdefault(key, default=None)</code>	Retorna valor existente o inserta <code>default</code> si faltaba.

Precisión: Las vistas de `keys()`, `values()` e `items()` reflejan cambios posteriores del diccionario; no son copias estáticas del contenido.

12. Mutabilidad, aliasing y copias

Muchos errores de Python no vienen de la sintaxis, sino de no distinguir entre compartir un objeto y copiarlo.

Mutabilidad

Tipo de objeto	Ejemplos	Consecuencia
Inmutable	int, float, bool, str, tuple, frozenset	No cambia in-place; una 'modificación' crea otro objeto o falla.
Mutable	list, dict, set, bytearray	Puede cambiar sin cambiar su identidad.

Aliasing

Aliasing ocurre cuando dos nombres apuntan al mismo objeto mutable. Cambiar el objeto a través de un nombre se observa también desde el otro.

Ejemplo mínimo

```
a = [1, 2]
b = a
b.append(3)
# a también refleja [1, 2, 3]
```

Copias superficiales y profundas

Concepto	Qué copia	Cuándo importa
Copia superficial	Crea una colección externa nueva, pero conserva referencias a los objetos internos.	Útil si los elementos internos no mutan o no necesitas aislarlos.
Copia profunda	Copia recursivamente los objetos contenidos cuando corresponde.	Útil en estructuras anidadas que deben independizarse.

Herramienta	Uso
<code>lista.copy()</code> / <code>lista[:]</code>	Copia superficial de lista.
<code>dict.copy()</code>	Copia superficial de diccionario.
<code>copy.copy(obj)</code>	Copia superficial genérica.
<code>copy.deepcopy(obj)</code>	Copia profunda genérica.

Precisión: Asignar no copia. La línea `b = a` crea otro nombre para el mismo objeto, no una nueva lista con contenido independiente.

13. Comprensiones, iterables e iteradores

Las comprensiones condensan patrones de transformación y filtrado. Para entenderlas bien conviene saber qué significa iterable e iterador.

Comprensiones

Tipo	Forma general
Lista	[expresion for variable in iterable if condicion]
Conjunto	{expresion for variable in iterable if condicion}
Diccionario	{clave: valor for variable in iterable if condicion}
Generador	(expresion for variable in iterable if condicion)

Iterable e iterador

Término	Definición operativa
Iterable	Objeto del que se puede obtener un iterador; se usa en <code>for</code> .
Iterador	Objeto que produce valores secuencialmente mediante <code>next()</code> y se agota.
Generador	Forma especial y cómoda de producir iteradores de manera perezosa.

Término técnico - perezoso: Una evaluación perezosa produce valores solo cuando se solicitan, en lugar de construir todos los resultados de una vez en memoria.

Funciones integradas útiles sobre iterables

Función	Uso
<code>iter(obj)</code>	Obtiene un iterador.
<code>next(it)</code>	Pide el siguiente valor del iterador.
<code>all(iterable)</code>	True si todos los elementos son verdaderos.
<code>any(iterable)</code>	True si al menos uno es verdadero.
<code>sorted(iterable)</code>	Retorna una lista ordenada nueva.

14. Funciones, parámetros y modularidad

Las funciones encapsulan lógica, reducen repetición y forman la unidad básica de reutilización dentro de un script o módulo.

Definición general

Estructura

```
def nombre_funcion(parametro_posicional, parametro_defecto=valor):
    # cuerpo
    return resultado
```

Parámetros y argumentos

Mecanismo	Descripción
Posicionales	Se asocian por orden.
Por palabra clave	Se asocian por nombre del parámetro.
Por defecto	Usan un valor predefinido si no se suministra argumento.
*args	Empaqueta argumentos posicionales extra en una tupla.
**kwargs	Empaqueta argumentos nombrados extra en un diccionario.
Parámetros solo posicionales /	Algunas firmas restringen que ciertos parámetros solo se pasen por posición.
Parámetros solo por palabra clave *	Permiten exigir argumentos explícitamente nombrados.

Precisión: Los parámetros con valor por defecto se evalúan al definir la función, no cada vez que se llama. Evita usar listas o diccionarios mutables como valores por defecto si esperas un objeto nuevo por llamada.

return, None y efectos secundarios

Caso	Qué ocurre
return valor	Termina la función y entrega un resultado.
return sin valor	Retorna None.
Sin return	Retorna None de forma implícita.
Efecto secundario	La función modifica estado externo, imprime, escribe archivos, altera objetos mutables, etc.

Docstrings y anotaciones

Una docstring es una cadena ubicada al inicio de una función, clase o módulo para documentar su propósito. Las anotaciones de tipo describen expectativas para lectores y herramientas, pero Python no las hace cumplir por sí mismo en tiempo de ejecución.

Lambda

lambda define funciones anónimas de una sola expresión. Son útiles para lógica breve, especialmente como clave de ordenamiento, pero una función def suele ser mejor cuando la intención necesita nombre, documentación o varias líneas.

15. Scope, namespaces y regla LEGB

El scope determina desde dónde puede verse un nombre. Los namespaces organizan asociaciones entre nombres y objetos.

Regla LEGB

Nivel	Significado	Ejemplo conceptual
L - Local	Nombres definidos dentro de la función actual.	Variables creadas en el cuerpo de una función.
E - Enclosed	Nombres de funciones externas envolventes.	Closures y funciones anidadas.
G - Global	Nombres del módulo actual.	Variables definidas fuera de funciones.
B - Built-ins	Nombres integrados disponibles por defecto.	len, print, int, ValueError.

Precisión: Las palabras clave del lenguaje, como if, for, class o return, no son nombres del namespace builtins. Son parte de la sintaxis del lenguaje.

global y nonlocal

Palabra	Efecto
global	Declara que las asignaciones a un nombre dentro de una función deben referirse al nivel del módulo.
nonlocal	Declara que las asignaciones deben apuntar a un nombre de una función externa envolvente.

Buena práctica: Usa global con moderación. En muchos diseños, pasar parámetros y retornar valores reduce acoplamiento y mejora la prueba del código.

16. Módulos, imports, paquetes y __main__

La modularidad permite separar código en archivos y reutilizarlo. Python trata cada archivo .py importable como módulo.

Módulos y namespaces

Un módulo tiene su propio namespace global. Al importar un módulo, sus nombres quedan disponibles mediante la notación modulo.nombre, salvo que se importen nombres concretos.

Sintaxis	Efecto
import modulo	Importa el módulo y exige acceso como modulo.nombre.
from modulo import nombre	Importa un nombre específico al namespace actual.
from modulo import *	Importa múltiples nombres; se evita en código mantenible por pérdida de claridad.
import modulo as alias	Asigna un alias local.

Paquetes

Un paquete organiza módulos en directorios para estructurar proyectos. En la práctica, permite jerarquías como paquete.subpaquete.modulo.

Bloque principal

Dunder main

```
if __name__ == '__main__':
    # código de ejecución directa
```

Cuando un archivo se ejecuta directamente, __name__ toma el valor '__main__'. Si se importa como módulo, __name__ contiene su nombre de módulo y el bloque no se ejecuta.

17. Programación orientada a objetos

La POO en Python organiza estado y comportamiento mediante clases. El lenguaje permite tanto diseños simples como jerarquías complejas, pero la claridad debe guiar la elección.

Clases, objetos y atributos

Concepto	Descripción
Clase	Plantilla que define atributos y métodos.
Objeto / instancia	Entidad concreta creada a partir de una clase.
Atributo	Dato asociado a una instancia o a la clase.
Método	Función asociada a una clase y normalmente invocada desde una instancia.

Concepto	Descripción
<code>__init__</code>	Inicializador que suele configurar el estado inicial al instanciar.

self, atributos de instancia y de clase

Elemento	Precisión
<code>self</code>	Nombre convencional del primer parámetro de métodos de instancia. Python no lo trata como palabra reservada.
Atributo de instancia	Se almacena por objeto, habitualmente como <code>self.atributo</code> .
Atributo de clase	Se define en el cuerpo de la clase y se comparte como atributo de la clase; puede sombreadse por instancia.

Herencia y super

La herencia permite reutilizar y especializar comportamiento. Python soporta herencia simple, múltiple y multinivel.

Concepto	Descripción
Sobrescritura	Una clase hija define un método con el mismo nombre que uno heredado.
<code>super()</code>	Retorna un proxy que continúa la búsqueda de métodos según el MRO, no simplemente 'el padre inmediato' en todos los casos.
MRO	Method Resolution Order: orden en el que Python busca atributos y métodos, especialmente relevante en herencia múltiple.

Término técnico - proxy: Un proxy es un objeto intermediario que representa o delega operaciones hacia otro comportamiento u objeto.

Polimorfismo y duck typing

Idea	Descripción
Polimorfismo	Objetos de distintos tipos responden a una misma operación o método de formas compatibles.
Duck typing	Se prioriza si el objeto ofrece el comportamiento requerido, no solo su clase nominal.

Métodos especiales y decoradores de clase

Elemento	Uso
@staticmethod	Función organizada dentro de una clase que no recibe self ni cls.
@classmethod	Método que recibe cls y opera a nivel de clase.
@property	Expone un método como atributo de lectura.
@propiedad.setter	Intercepta asignaciones para validar o controlar escritura.
@propiedad.deleter	Controla la eliminación mediante del objeto.propiedad.

Dunder methods frecuentes

Método	Interviene en
__str__(self)	Representación amigable con str() o print().
__repr__(self)	Representación orientada a depuración.
__eq__(self, other)	Operador ==.
__lt__ / __gt__	Comparaciones < y >.
__add__(self, other)	Operador +.
__contains__(self, item)	Operador in.
__getitem__(self, key)	Acceso con corchetes.
__len__(self)	len(obj).
__iter__(self)	Obtención de iterador.

Encapsulación por convención

Python no impone privacidad estricta como otros lenguajes. Un nombre con prefijo `_` atributo comunica uso interno por convención. El prefijo `__` atributo activa name mangling, una transformación de nombre diseñada para evitar colisiones en clases, no para seguridad absoluta.

18. Excepciones y gestión de errores

Las excepciones representan condiciones anómalas que aparecen en ejecución. Gestionarlas bien evita programas frágiles y mensajes confusos.

Estructura

Forma general

```
try:
    # código que podría fallar
except ExcepcionEspecifica as error:
    # manejo controlado
```

```

else:
    # ocurre si try no lanzó excepción
finally:
    # limpieza que se ejecuta siempre

```

Excepciones comunes

Excepción	Ejemplo típico
ValueError	int('pizza').
TypeError	Operación entre tipos incompatibles.
ZeroDivisionError	Dividir entre cero.
IndexError	Acceder a una posición inexistente en una secuencia.
KeyError	Acceder con dict[clave] cuando falta la clave.
AttributeError	Solicitar un atributo o método inexistente.
NameError	Usar un nombre no definido.
FileNotFoundError	Abrir una ruta que no existe.
PermissionError	Operación bloqueada por permisos del sistema.

raise y excepciones propias

raise permite lanzar una excepción deliberadamente. También es posible definir excepciones personalizadas heredando de Exception, útil cuando un dominio necesita errores semánticos propios.

Precisión: Captura excepciones específicas cuando sea posible. Un except demasiado amplio puede esconder errores reales y dificultar la depuración.

19. Archivos, rutas y sistema operativo

Trabajar con archivos implica abrir recursos del sistema, leer o escribir y garantizar cierre correcto. El administrador de contexto with es la forma recomendada.

with open

Lectura segura

```

with open('archivo.txt', 'r', encoding='utf-8') as archivo:
    contenido = archivo.read()

```

Modos de apertura

Modo	Efecto
'r'	Lectura de archivo existente.
'w'	Escritura; crea o reemplaza el contenido existente.
'a'	Anexado al final; crea si no existe.
'x'	Creación exclusiva; falla si ya existe.
'b'	Se añade para modo binario: rb, wb, etc.
'+'	Se combina para lectura y escritura: r+, w+, a+.

Métodos de archivo

Método	Uso
read()	Lee todo el contenido.
readline()	Lee una línea.
readlines()	Lee líneas en una lista.
write(texto)	Escribe texto o bytes según el modo.
writelines(iterable)	Escribe una secuencia de strings; no añade saltos automáticamente.

Precisión: En archivos de texto conviene especificar `encoding='utf-8'` salvo que sepas que necesitas otro. Para binarios, usa modo `'b'`.

Rutas: os.path y pathlib

Herramienta	Idea
os.path.exists(path)	Comprueba existencia.
os.path.isfile(path)	Comprueba si es archivo.
os.path.isdir(path)	Comprueba si es directorio.
pathlib.Path	Interfaz orientada a objetos para rutas, cómoda en código moderno.

20. Módulos estándar clave

La biblioteca estándar amplía Python sin instalar paquetes externos. Esta sección resume módulos que suelen aparecer temprano en proyectos y cursos.

math

Aporta funciones y constantes matemáticas para operaciones reales frecuentes: raíces, trigonometría, logaritmos, redondeos de piso y techo, pi y e.

json

Función	Uso
<code>json.dump(obj, fp, indent=None)</code>	Serializa un objeto de Python a un archivo JSON.
<code>json.load(fp)</code>	Lee JSON desde un archivo y lo convierte a objetos de Python.
<code>json.dumps(obj)</code>	Serializa a una cadena.
<code>json.loads(texto)</code>	Deserializa desde una cadena.

Término técnico - serializar: Serializar es convertir una estructura de datos a un formato apto para guardarse o transmitirse. Deserializar es reconstruir la estructura a partir de ese formato.

Precisión: Al procesar JSON no confiable, el tamaño y complejidad del dato importan: un archivo malicioso o excesivo puede consumir demasiados recursos.

csv

Herramienta	Uso
<code>csv.reader(csvfile)</code>	Lee filas como listas de strings.
<code>csv.writer(csvfile)</code>	Escribe filas en formato CSV.
<code>writerow(row)</code>	Escribe una fila.
<code>DictReader / DictWriter</code>	Lectura y escritura orientadas a diccionarios por columna.

Precisión: Para archivos CSV, la documentación recomienda abrir con `newline=""` y controlar el encoding cuando corresponda.

time

Función	Uso
<code>time.sleep(secs)</code>	Suspende el hilo actual al menos aproximadamente el tiempo pedido; la pausa puede durar más por planificación del sistema.
<code>time.time()</code>	Tiempo de reloj como timestamp flotante.
<code>time.perf_counter()</code>	Reloj de alta resolución útil para medir duraciones.

datetime

Elemento	Uso
<code>datetime.date(year, month, day)</code>	Representa fecha.
<code>datetime.date.today()</code>	Fecha local actual.
<code>datetime.time(...)</code>	Representa hora independiente de fecha.
<code>datetime.datetime.now()</code>	Fecha y hora locales actuales.
<code>obj.strftime(formato)</code>	Convierte fecha/hora a texto formateado.
<code>timedelta</code>	Representa duraciones y diferencias.

Término técnico - naive y aware: Un `datetime` naive no contiene información de zona horaria; un `datetime` aware sí lleva contexto de zona u offset y representa momentos de forma menos ambigua.

threading

Elemento	Uso
<code>threading.Thread(target, args=())</code>	Crea un hilo para ejecutar una función objetivo.
<code>thread.start()</code>	Inicia el hilo.
<code>thread.join()</code>	Espera a que el hilo termine.

`threading` es útil para concurrencia, especialmente tareas de espera o I/O. En implementaciones habituales de CPython, el GIL limita el paralelismo efectivo para tareas intensivas de CPU; existen builds `free-threaded` modernos donde esa restricción puede desactivarse, pero no son la configuración por defecto.

Término técnico - concurrencia: Concurrencia significa que varias tareas progresan en periodos superpuestos. Paralelismo significa que se ejecutan físicamente al mismo tiempo.

21. Buenas prácticas y trampas frecuentes

Esta sección condensa decisiones que evitan errores comunes y mejoran legibilidad sin convertir el manual en una guía de estilo extensa.

Situación	Recomendación
Comparar con <code>None</code>	Usa <code>is None</code> o <code>is not None</code> .
Ordenar listas	Recuerda que <code>list.sort()</code> modifica y retorna <code>None</code> ; <code>sorted()</code> crea nueva lista.
Valores por defecto mutables	Evítalos en firmas de funciones; usa <code>None</code> y crea el objeto dentro.

Situación	Recomendación
Strings en bucles grandes	Construir con concatenación repetida puede ser menos conveniente; para muchas piezas, considera <code>join()</code> .
Validación de input	Convierte dentro de <code>try/except</code> específico.
Archivos de texto	Usa <code>with</code> y, cuando aplique, <code>encoding='utf-8'</code> .
Dicts	Usa <code>get()</code> si la clave puede faltar y la ausencia no debe ser excepcional.
Sets	Úsalos para pertenencia rápida y eliminación de duplicados, no para conservar orden semántico.
Copias	Distingue asignación, copia superficial y copia profunda.
Imports	Prefiere importaciones explícitas a <code>import *</code> .

22. Glosario técnico

Definiciones breves de términos que aparecen a lo largo del manual y que conviene dominar para leer documentación con soltura.

Término	Definición
Abstracción	Modelo simplificado que conserva lo importante de una entidad o problema y oculta detalles innecesarios para cierto nivel de análisis.
Alias	Segundo nombre que referencia el mismo objeto que otro nombre.
Argumento	Valor concreto entregado a una función al llamarla.
Atributo	Nombre asociado a un objeto o clase que representa dato o comportamiento accesible con punto.
Byte	Unidad de ocho bits usada para representar datos binarios.
Code point	Número asignado por Unicode a una unidad abstracta de texto.
Codificación	Reglas que transforman texto Unicode en bytes y viceversa.
Concurrencia	Organización de varias tareas que avanzan en periodos superpuestos.
Descriptor de archivo	Recurso del sistema que representa un archivo abierto o canal de entrada/salida.
Duck typing	Enfoque donde importa que un objeto ofrezca el comportamiento esperado, no solo su tipo nominal.
Efecto secundario	Cambio observable fuera del valor retornado: modificar un objeto, imprimir, escribir un archivo, etc.
Encapsular	Agrupar estado y comportamiento detrás de una interfaz controlada.

Término	Definición
Excepción	Evento anómalo de ejecución que puede propagarse o capturarse.
Float	Tipo numérico de punto flotante; representa aproximaciones finitas de muchos valores reales.
Glifo	Forma visual que una fuente usa para dibujar un carácter o combinación de caracteres.
Hash	Valor numérico calculado a partir de un objeto para apoyar estructuras como dict y set.
Hashable	Objeto cuyo hash se mantiene estable y que puede utilizarse como clave de dict o elemento de set.
Identidad	Condición de ser exactamente el mismo objeto en memoria, no solo tener valor equivalente.
In-place	Operación que modifica el objeto existente en lugar de construir y retornar otro resultado principal.
Inmutable	Objeto cuyo contenido no puede modificarse después de creado.
Iterable	Objeto que puede recorrerse en un for porque produce o permite obtener un iterador.
Iterador	Objeto que entrega elementos uno por uno y conserva su estado de avance.
MRO	Method Resolution Order: orden de búsqueda de atributos y métodos en jerarquías de clases.
Mutable	Objeto cuyo contenido puede modificarse después de creado.
Namespace	Espacio que asocia nombres con objetos.
Objeto	Entidad con identidad, tipo y valor dentro del modelo de Python.
Parámetro	Nombre declarado en la firma de una función para recibir un argumento.
Punto flotante	Representación numérica finita que aproxima cantidades reales usando mantisa y exponente.
Proxy	Intermediario que delega operaciones o acceso hacia otro comportamiento.
Referencia	Vínculo mediante el cual un nombre apunta a un objeto.
Serializar	Convertir datos a una representación almacenable o transmisible.
Scope	Región o contexto donde un nombre es visible y resoluble.
Truncar	Eliminar la parte fraccionaria sin redondear.
Unicode	Estándar que organiza la representación de texto de múltiples lenguas y símbolos.

Término	Definición
Wrapper	Función o capa envolvente que añade o modifica comportamiento alrededor de otra función u objeto.

23. Fuentes oficiales consultadas

Las precisiones técnicas de este manual se contrastan con documentación oficial de Python y con el Unicode HOWTO. Esta lista funciona como guía para profundizar.

Fuente	Temas contrastados
Python Standard Library - Built-in Types	Tipos integrados, verdad lógica, secuencias, listas, sets, diccionarios y comparaciones.
Python Standard Library - Built-in Functions	int, round, super, input, format, enumerate, zip, sorted y otras funciones.
Python Tutorial - Classes	self, clases, herencia y fundamentos de POO.
Python Reference - Data Model	Diccionarios, orden de inserción y aspectos del modelo de objetos.
Python Reference - Expressions	Operadores in, not in, is e is not.
Python Tutorial - Modules	Imports, namespaces de módulos y buenas prácticas.
Python Tutorial - Errors and Exceptions	try, except, else, finally, raise y limpieza de recursos.
Python Tutorial - Input and Output	open(), modos, archivos de texto, encoding y binarios.
Python Library - json	Serialización y advertencias de procesamiento.
Python Library - csv	Lectura y escritura de CSV.
Python Library - datetime	Fechas, horas y distinción naive/aware.
Python Library - time	sleep y medición temporal.
Python Library - threading	Hilos, GIL y builds free-threaded.
Python Library - copy	Copia superficial y copia profunda.
Python Unicode HOWTO	Unicode, code points, glyphs, bytes y UTF-8.

Cierre: Un buen manual de referencia no intenta sustituir la documentación oficial: la ordena, la compacta y la vuelve más consultable para una necesidad concreta.